

Foundations of Artificial Intelligence

Part IV: Machine Learning

Department of Computer Science

Lecture Roadmap

1. What Is Learning?
2. Supervised vs Unsupervised Learning
3. Training vs Testing
4. Overfitting & Underfitting
5. Linear Regression
6. k-Nearest Neighbours (k-NN)
7. Decision Trees
8. k-Means Clustering

Learning Objectives

By the end of this lecture, you should be able to:

- Explain what **learning** means in the context of AI.
- Distinguish **supervised** from **unsupervised** learning.
- Understand the roles of **training** and **testing** data.
- Recognise and reason about **overfitting** and **underfitting**.
- Describe four fundamental algorithms: Linear Regression, k-NN, Decision Trees, k-Means.

What Does It Mean To Learn?

“A computer program is said to learn from experience E with respect to tasks T and performance measure P , if its performance on T improves with E .”

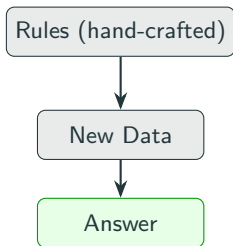
— Tom Mitchell, 1997



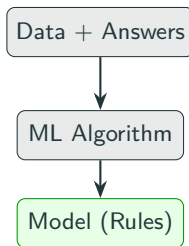
Unlike classical AI (hand-coded rules), ML **discovers** patterns automatically from data.

Classical AI vs Machine Learning

Classical / Rule-Based AI



Machine Learning



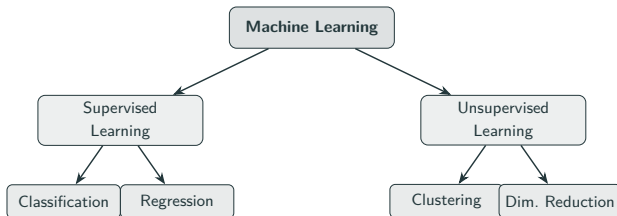
ML inverts the classical pipeline: we provide data and desired outputs; the algorithm finds the mapping.

Why Machine Learning Now?

1. **Big Data:** billions of digital interactions every day.
2. **Compute:** GPUs, cloud, specialised hardware.
3. **Algorithms:** decades of research reaching maturity.
4. **Open ecosystems:** Python, scikit-learn, PyTorch, etc.

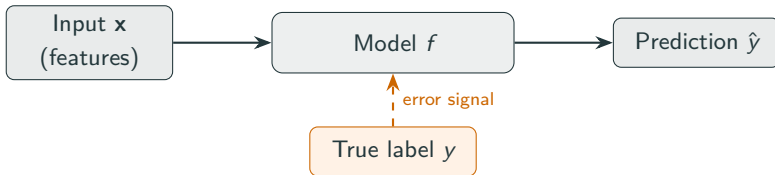
Real applications: spam filtering, product recommendations, medical diagnosis, fraud detection, voice assistants.

The Two Main Paradigms



Supervised Learning

- Every training example comes with a **label** (the correct answer).
- Goal: learn a function f such that $f(\mathbf{x}) \approx y$.



Examples: email spam (classification), house price prediction (regression).

Unsupervised Learning

- Data has **no labels** — we do not know the right answer in advance.
- Goal: discover **hidden structure** in the data.



Raw data (unlabelled) → clusters emerge

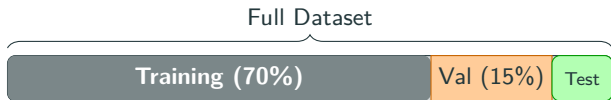
Examples: customer segmentation, anomaly detection, topic modelling.

Supervised vs Unsupervised: Quick Comparison

Aspect	Supervised	Unsupervised
Labels	Required	Not needed
Goal	Predict output	Discover structure
Evaluation	Compare to labels	Harder; domain judgement
Examples	Spam filter, diagnosis	Customer segments, anomaly

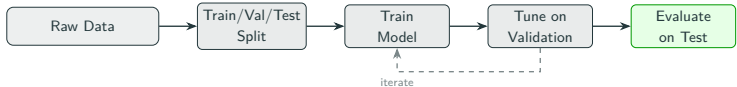
The Data Split

We never evaluate a model on data it learned from — that would be a cheat.



- **Training set:** used to fit model parameters.
- **Validation set:** used to tune *hyperparameters* (e.g., model complexity).
- **Test set:** held back; used *once* to estimate real-world performance.

The Training-Testing Pipeline

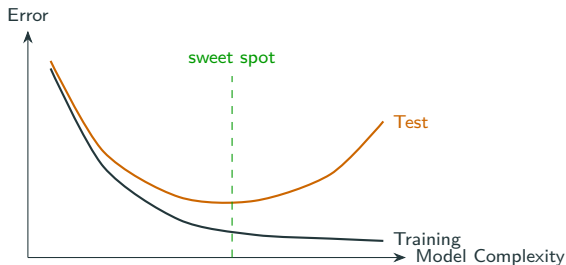


*The test set must not influence any design decision.
Leaking it leads to over-optimistic reported performance.*

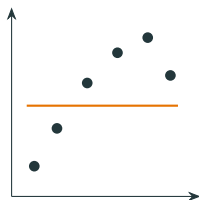
Generalisation: The Core Goal

- **Training error:** how well the model fits the data it has seen.
- **Test (generalisation) error:** how well it performs on *new*, unseen data.

We want **low generalisation error**, not just low training error.

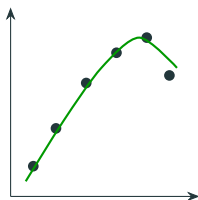


The Goldilocks Problem



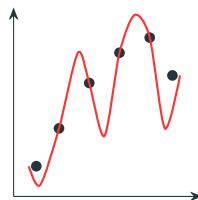
Underfitting

Too simple



Good Fit

Just right



Overfitting

Too complex

Underfitting

- The model is **too simple** to capture the true pattern.
- **High training error** and high test error.
- Also called **high bias**.

Causes:

- Model too constrained (e.g., fitting a straight line to curved data).
- Too few features provided to the model.
- Insufficient training.

Fix: use a more expressive model or add useful features.

Overfitting

- The model **memorises** training data including its noise.
- **Low training error** but high test error.
- Also called **high variance**.

Causes:

- Model too complex relative to the amount of data.
- Too many features or training for too long.

Fixes: more data, simpler model, regularisation, early stopping, cross-validation.

Bias–Variance Trade-off (Summary)

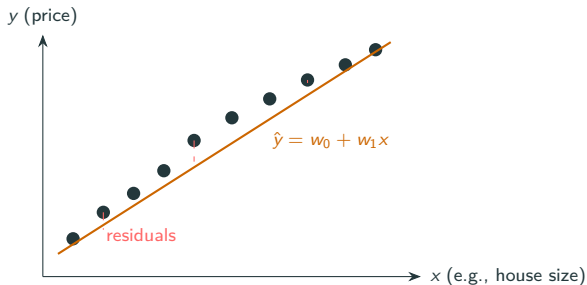
	Bias	Variance	Result
Underfitting	High	Low	Bad on train & test
Good fit	Low	Low	Good generalisation
Overfitting	Low	High	Good train, bad test

Total Error $\approx$ Bias² + Variance + Irreducible Noise

Linear Regression: The Idea

Task: predict a *continuous* output from one or more input features.

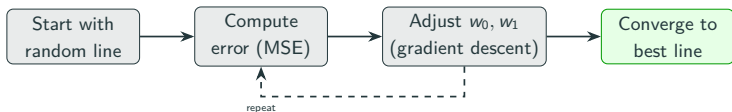
Intuition: fit the best **straight line** through the data.



How Does It Learn?

The model minimises the **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Intuition: if the line predicts 300k but reality is 350k, nudge the slope upward.

Linear Regression: Key Points

Strengths

- Fast to train and interpret.
- Works well when the true relationship is approximately linear.
- Each weight w_i = effect of feature i .

Limitations

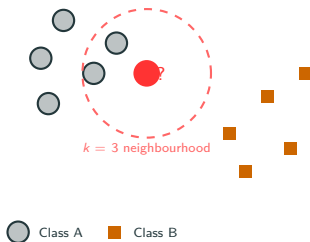
- Assumes linearity — can underfit curved patterns.
- Sensitive to outliers.
- Not suited for classification targets.

Real-world uses: sales forecasting, energy demand prediction, insurance pricing.

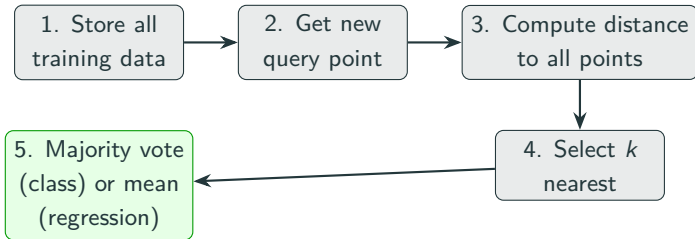
k-NN: The Idea

Principle: *"You are judged by the company you keep."*

To classify a new point, look at the k closest training points and take a **majority vote**.



k-NN: How the Algorithm Works



Common distance measure: **Euclidean distance**

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_i (a_i - b_i)^2}$$

Choosing k : Impact on Decision Boundary

Small k (e.g., $k = 1$):

- Very wiggly boundary.
- Sensitive to noise.
- Risk of **overfitting**.

Large k (e.g., $k = N$):

- Very smooth boundary.
- Ignores local structure.
- Risk of **underfitting**.

Typical starting point: try $k = \sqrt{N}$ and tune via cross-validation.

k-NN: Key Points

Strengths

- No training phase (*lazy learner*).
- Naturally handles multi-class.
- Simple to understand.

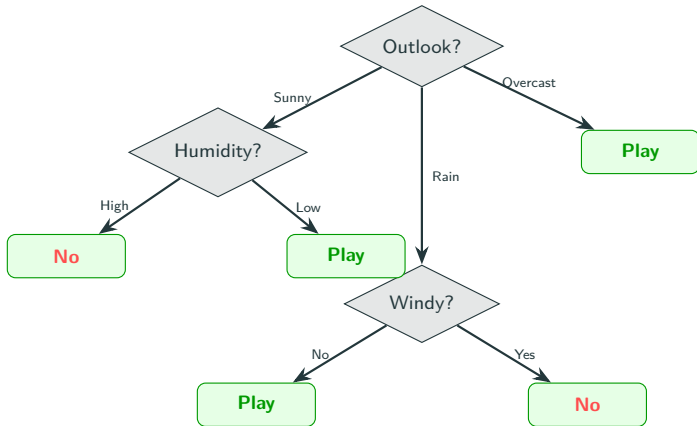
Limitations

- Slow at prediction time for large datasets.
- Suffers in high dimensions (*curse of dimensionality*).
- Memory-intensive — stores all data.

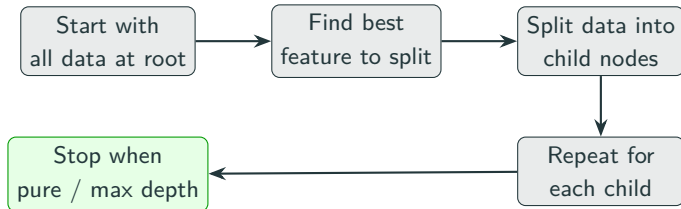
Real-world uses: recommendation systems, image recognition, anomaly detection.

Decision Trees: The Idea

Model decisions as a **tree of if-then rules**. Each internal node tests a feature; each leaf gives a prediction.



How a Decision Tree Is Built



Best split = the feature/threshold that most reduces **impurity**.
Common measures: *Gini impurity*, *Information Gain (entropy)*.

Gini Impurity: Intuition

A node is **pure** if all examples belong to one class.

$$\text{Gini} = 1 - \sum_c p_c^2$$



We choose the split that *maximises purity* of the resulting child nodes.

Decision Trees: Key Points

Strengths

- Highly interpretable — can draw the tree.
- Handles mixed feature types.
- No feature scaling required.

Limitations

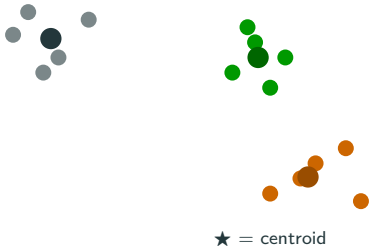
- Prone to overfitting (deep trees).
- Unstable — small data changes can flip the tree.
- Axis-aligned splits only.

Extensions overcome limitations: **Random Forests** (many trees + voting), **Gradient Boosting**.

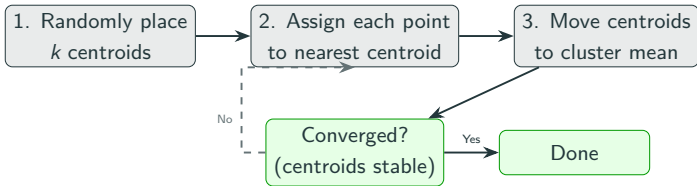
Real-world uses: medical triage, loan approval, customer churn diagnosis.

k-Means: The Idea

Unsupervised. Group n data points into k clusters such that points within a cluster are **as similar as possible**.



k-Means: The Algorithm



k-Means: Step-by-Step Visualisation

Step 1: Initialise



Step 2: Assign

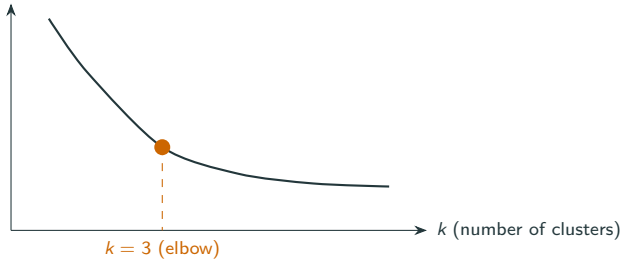


Step 3: Update



Choosing k : The Elbow Method

Within-cluster sum of squares (WCSS)



At the elbow, adding more clusters gives *diminishing returns* in compactness.

k-Means: Key Points

Strengths

- Simple and fast.
- Scales to large datasets.
- Easy to interpret clusters.

Limitations

- Must choose k in advance.
- Sensitive to initialisation and outliers.
- Assumes spherical, equally sized clusters.

Real-world uses: customer segmentation, image compression, document grouping, geo-spatial analysis.

Algorithm Comparison at a Glance

Aspect	Lin. Reg.	k-NN	Dec. Tree	k-Means
Type	Supervised	Supervised	Supervised	Unsupervised
Task	Regression	Both	Both	Clustering
Interpretable	Yes	Moderate	Yes	Moderate
Training speed	Fast	Instant	Moderate	Moderate
Prediction speed	Fast	Slow	Fast	—
Handles non-linear	No	Yes	Yes	—

Assessment Q1 — What is Learning?

Question: Explain the difference between a hand-coded rule system and a machine learning model using a spam filter as your example.

Expected points:

- Rule system: engineer writes rules (“if subject contains ‘free money’ → spam”).
- ML: algorithm infers rules from thousands of labelled emails.
- ML adapts when spammers change tactics; rules must be updated manually.

Assessment Q2 — Supervised vs Unsupervised

Classify each scenario (with justification):

1. Predicting whether a loan applicant will default.
2. Grouping hospital patients by symptom similarity (no diagnosis yet).
3. Estimating tomorrow's electricity demand from historical usage.
4. Detecting unusual network traffic with no prior examples of attacks.

A model achieves 99% accuracy on training data and 61% on test data.

- What is this phenomenon called?
- What are two likely causes?
- Propose two concrete remedies.

Assessment Q4 — Algorithm Choice

For each task below, state which algorithm you would *first try* and why:

1. Predicting the sale price of a house given its features.
2. Classifying handwritten digits (0–9).
3. Segmenting 10,000 retail customers into groups with no prior labels.
4. Determining whether a tumour is malignant based on biopsy features.

Scenario: You train a decision tree and notice it perfectly classifies all training data but performs poorly on validation data.

- Draw (or describe) what an over-grown decision tree looks like.
- What does *pruning* mean and how does it help?
- How would using a **Random Forest** address this issue?

Key Takeaways

1. **Learning** = improving performance on tasks through experience (data).
2. **Supervised learning** uses labelled data; **unsupervised** finds hidden structure.
3. Always split data into **train** / **validation** / **test** — never evaluate on training data.
4. **Overfitting** memorises noise; **underfitting** misses the signal. Tune complexity.
5. Four core algorithms:
 - **Linear Regression** — predict continuous values with a line.
 - **k-NN** — classify by neighbour majority vote.
 - **Decision Trees** — hierarchical if-then rules.
 - **k-Means** — find clusters by iterating assignments and centroids.

Questions?